7 Best Practices for API Design 🔥

#86: Break into API design (8 Minutes)



NEO KIM
SEP 07, 2025

128 6 15

Get my system design playbook for FREE on newsletter signup:

Type your email... Subscribe

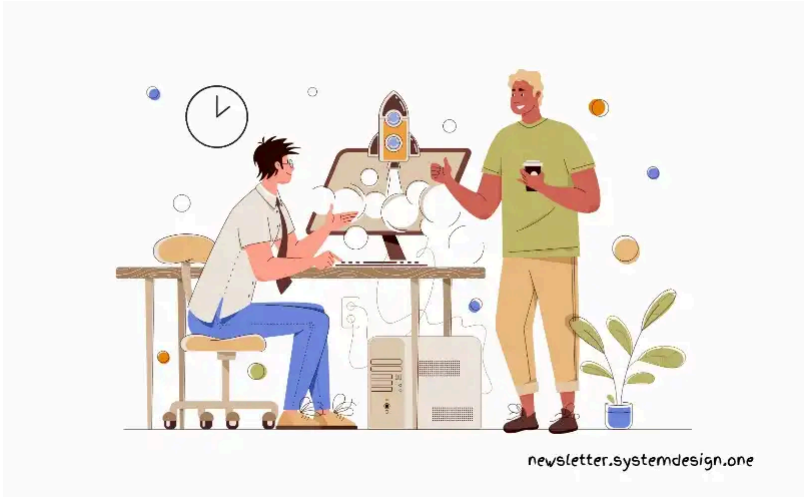
This post outlines best practices for API design. You will find references at the botto this page if you want to go deeper.

- *Share this post & I'll send you some rewards for the referrals.*

Once upon a time, there was a tiny startup.

They had only a few customers.

So they ran their site using a simple monolith architecture.



But one day, their site became extremely popular.

So they set up a microservices architecture for scalability.

Yet they didn't know much about API design.

And their public APIs often failed from spiky traffic.

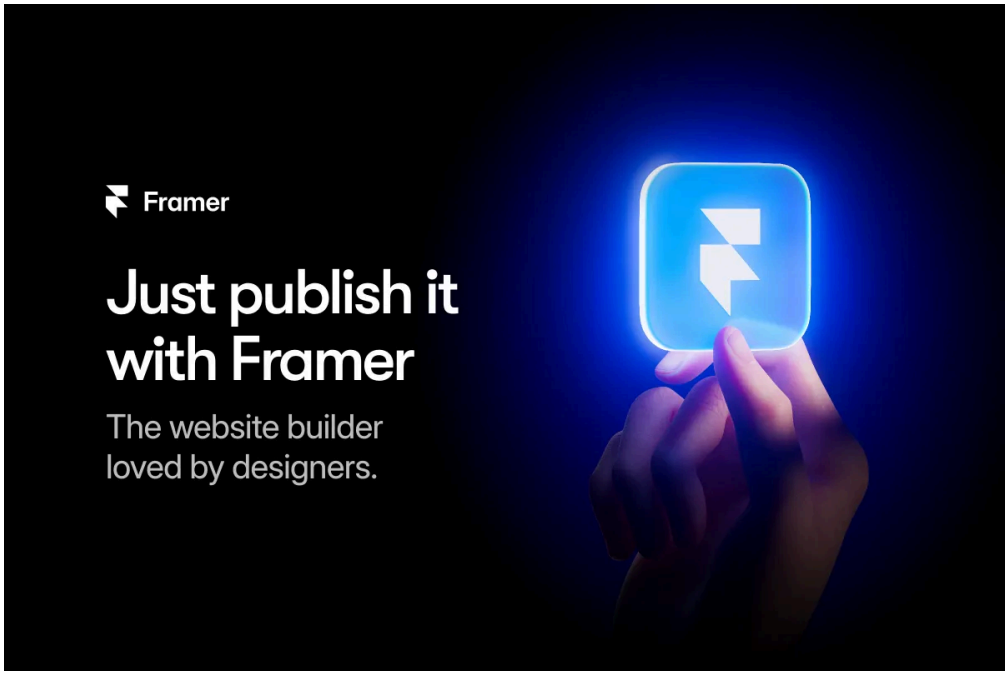
Also they sent the entire dataset when users asked for just the latest data.

Besides it became difficult to manage APIs as they added more features.

So they decided to study the best practices to design APIs.

Onward.

Design, Animate, Publish—All in Framer (Sponsor)



Still using a copy-paste website? **Framer** is the design-first, no-code website buil that lets anyone ship a production-ready site in minutes. Whether you're starting a template or a blank canvas, Framer gives you total creative control—no coding

required. Add animations, localize with one click, and collaborate in real-time wit whole team. You can even A/B test and track clicks with built-in analytics.

Ready to build a site that looks hand-coded—without hiring a developer? Launch site for free at Framer dot com, and use code **SYSTEMDESIGN** for a free month Framer Pro.

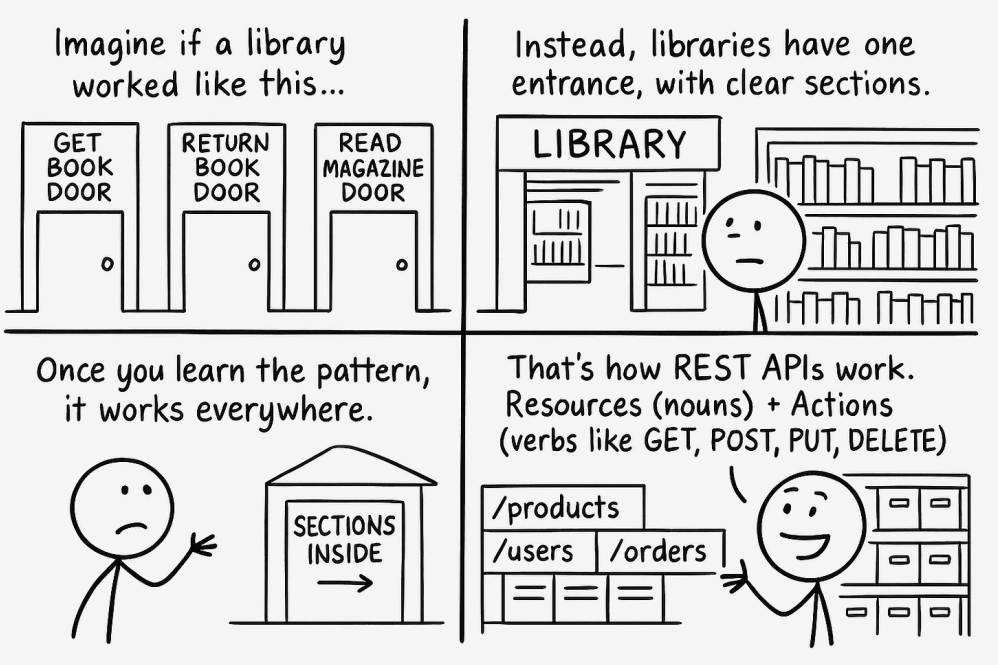
Best Practices for API Design

Here's what they learned:

1. REST Fundamentals

REST stands for Representational State Transfer.

It's an architectural pattern for systems to talk with each other over the Internet. helps to organize the data simply and clearly.

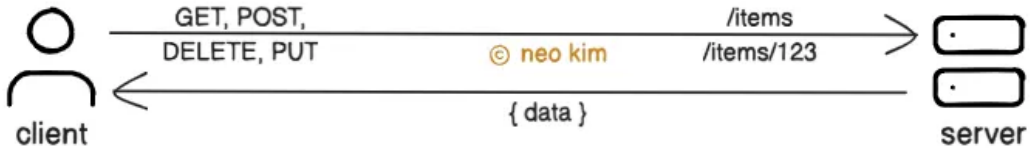


REST Explained With an Analogy

Think of REST like a book library.

There is just one entrance to borrow or return books. While books get organized different sections by subject.

REST APIs organize data into resources similarly.



How REST API Works

Here's how it works:

- Each data resource gets represented by an API endpoint.
  - For example, `/items` represents a collection of items.
  - While `/items/<item>` represents a specific item.
- The standard HTTP methods allow interaction with the data.
  - `GET`: lets you read data from the resource.
  - `POST`: lets you add a new item to the resource.
  - `PUT`: lets you replace an item with newer data.
  - `DELETE`: lets you remove an item from the resource.

This approach makes the APIs predictable and easy to interact with.

Yet it might be limiting for some real-world actions, such as publishing a draft of document. So it's necessary to have a balance between REST principles and a pragmatic approach.

Let's keep going!

2. Error Handling

A status code tells the client whether a request succeeded or failed.

API Error handling means returning clear, consistent error messages if something wrong. Thus making it easy for the client to handle the error properly.

Imagine you're at an airport.

A bad API error is like the departure board displaying random errors when flight changes. It doesn't tell you which flight, why, or what to do next.

While a good API error is like displaying the flight name and its reason behind tir changes.



HTTP Status Codes

Here's how it works:

- The response status code tells whether a client error or a server error occurre
- And response body includes extra details about what went wrong.

Proper error handling in APIs makes a site reliable.

Yet it needs extra effort and discipline to handle each failure scenario. Besides err message must include only enough information to avoid security risks.

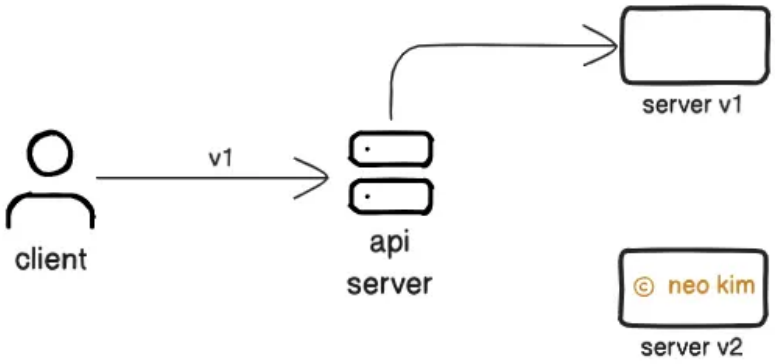
Ready for the next technique?

3. API Versioning

It's necessary to version an API so new changes don't break existing clients.

Think of API versioning like publishing a new book edition.

The old book copies that have already been sold don't get updated. But the new edition gets released with improvements. So readers who need the updates can g the latest edition. And those using the old one can read it without issues.



How API Versioning Works

Here's how it works:

- The API URL path includes the version number.
  - First version → `https://app.com/v1/subpath`
  - Second version → `https://app.com/v2/subpath`
- The old version doesn't get updated, while new changes happen on the lates version.

Thus ensuring backward compatibility.

Here are 2 alternative approaches to implement API versioning:

- HTTP headers

- Client includes the version in the Accept header.
- `Accept: application/vnd.api.v1+json`
- It means API version 1 in JSON format.

- Query parameters
  - Client includes the version parameter in the URL.
  - `https://app.com/subpath?version=1`
  - It means API version 1.

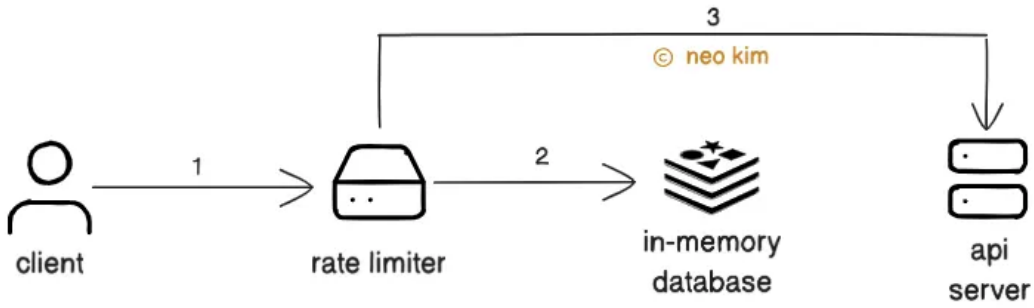
But query parameters are designed for filtering or searching data. So it's better to avoid them for versioning.

API versioning makes a site reliable. Yet it increases the complexity and maintenance efforts. So use it only for breaking changes.

4. Rate Limiting

Rate limiting means controlling the number of requests a client can make to an API within a period. It prevents server overload and protects the API from abuse.

Imagine rate limiting as telling a person how many things they can have during a giveaway.



How Rate Limiter Works

Here's how it works:

1. The client sends requests to the server through the rate limiter.

2. The rate limiter tracks the number of requests from each client using a count
3. The rate limiter rejects a request in case it exceeds the limit.

The API includes special HTTP headers in the response about the rate-limiting ru

Here are some of them:

- `X-RateLimit-Limit` → total requests allowed.
- `X-RateLimit-Remaining` → requests left before being rate-limited.
- `X-RateLimit-Reset` → time when the rate-limiting counter resets.

While the API returns a `429 Too Many Requests` status code after the client exc the limit.

Rate limits ensure the high availability of a site. But it's necessary to set a reasona rate limit for a better user experience.

Besides many people might share the same IP address. So it's better to rate limit person using an API key instead of their IP address for accuracy and fairness.

Ready for the best part?

5. Pagination

API pagination is the technique of breaking a large dataset into smaller chunks.

It lets the client request data in smaller parts instead of all at once. Thus achievin latency and reduced bandwidth usage.

Imagine a big book with 1,000 pages. Pagination is like reading through it one pa a time, instead of all at once.

There are 2 ways to implement pagination:

Offset Pagination

It's the most common technique for interacting with a dataset in pages.

Here's how it works:

1. The server queries the database to find the total number of items and pages

```
SELECT COUNT (*) AS total
FROM items;
```

2. The client specifies the page number and the number of items per page in th request.
   - For example, `https://app.com/subpath?page=3&limit=20` → skip the 40 items and return the next 20

```
SELECT *
FROM items
ORDER BY created_at DESC
LIMIT 20 OFFSET 40;
```

3. The server responds along with pagination metadata.

```
{
    "data": [{...}],
    "paging": {
        "total": 300,   // total items
        "page": 3,      // current page
        "pages": 15     // total pages
    }
}
```

Offset pagination is simple to understand and easy to implement. Also it lets the access random pages.

But it's slow with large datasets because the database scans the skipped records well. Besides there's a risk of inconsistent results if data gets added or removed d paging.

Cursor Pagination

It uses a pointer (**cursor**) to a specific record in the dataset. It means the server re results after the cursor instead of skipping records.

Here's how it works:

1. A unique sequential field, such as ID or timestamp, gets chosen as the cursor
2. The client then includes the cursor parameter in the request.
    - `https://app.com/subpath?cursor=abc&limit=20`
3. The server fetches the next 20 items after the cursor using the database inde

```
SELECT *
FROM items
WHERE created_at < %cursor
ORDER BY created_at DESC
LIMIT 21; -- one extra to find the next cursor
```

4. The server includes the new cursor as well in the response for future request

```
{
    "data": [{...}],
    "next_cursor": "cN315672McETHy"
}
```

Cursor pagination doesn't scan skipped records; instead, it uses the database ind Thus making it efficient for large datasets.

Also it gives consistent results even if data gets added or removed during paging it's complex to implement and doesn't support navigation to a specific page.

So choose the pagination technique based on the data size and data update frequency.

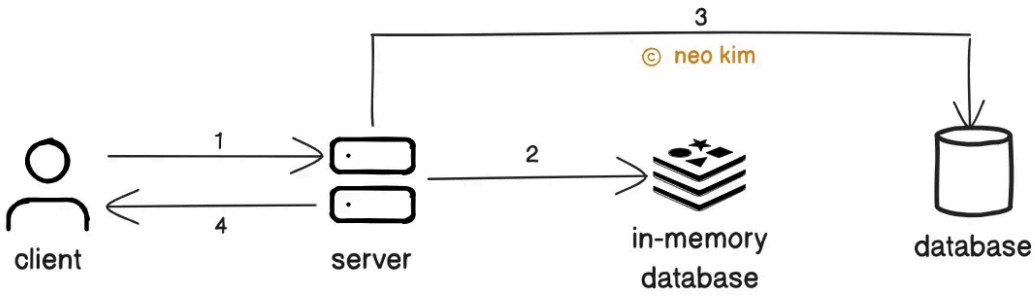
Ready for the best technique?

6. Idempotency

Idempotency means processing a request only once to avoid unwanted side effe

Imagine paying for something online. And mistakenly getting charged twice for t same transaction.

An idempotent API prevents this problem.



How Idempotent API Works

Here's how it works:

1. The client generates a unique string (UUID) to use as the idempotency key.
2. The client includes this key in the HTTP request header.
3. The server processes the request and caches its response.
4. Then it stores the idempotency key on an in-memory database.

And the in-memory database gets queried to check if a future request has been processed already.

Also the client generates a new UUID whenever the request payload changes. Th ensuring only new requests get processed.

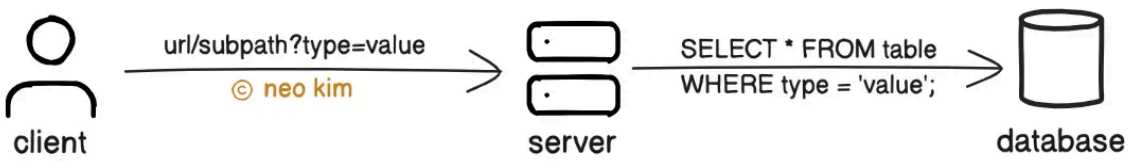
Here are 2 alternative approaches to implement idempotency:

- Use database constraints, such as unique keys, to prevent duplicates.
- Use a message queue with built-in deduplication.

Idempotency is essential for building reliable APIs. Yet it increases the complexity operational costs. Besides the idempotency keys should be removed periodically reduce memory usage. So use it mainly for critical APIs where retries could cause unwanted side effects.

7. Filtering and Sorting

API filtering means returning results that match specific conditions in request parameters.

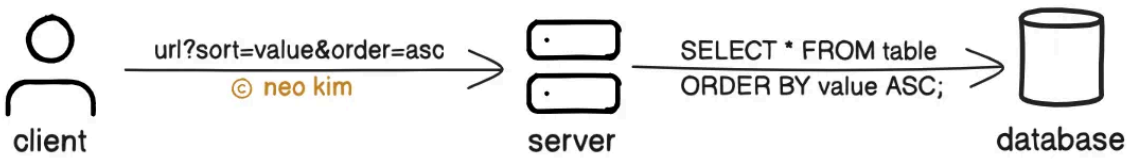


How API Filtering Works

Here's how it works:

1. The client includes filter parameters in the request URL.
    - `https://app.com/subpath?type=value`
2. The server reads the parameters.
3. Then it queries the database with those conditions.
4. The server returns only the matching records.

Filtering improves performance by reducing bandwidth usage. But it increases th complexity of server logic. So use it only if specific data from a large dataset is ne



How API Sorting Works

API sorting means ordering the results by a specific field, using request paramete

Here's how it works:

1. The client includes sorting parameters in the request URL.
    - `https://app.com/subpath?sort=value&order=asc`
2. The server reads those parameters.
3. It then creates a database query with the ORDER BY clause.
4. The server returns the results sorted by the requested field.

Sorting improves usability by ordering data. But it might slow down queries on la datasets without indexes. So use it only if ordering is important.

The internet runs using APIs.

While good APIs are consistent, predictable, and designed to scale without probl

So it's necessary to have clear documentation, useful error messages, and proper monitoring for APIs.

Subscribe to get simplified case studies delivered straight to your inbox:

Type your email... Subscribe

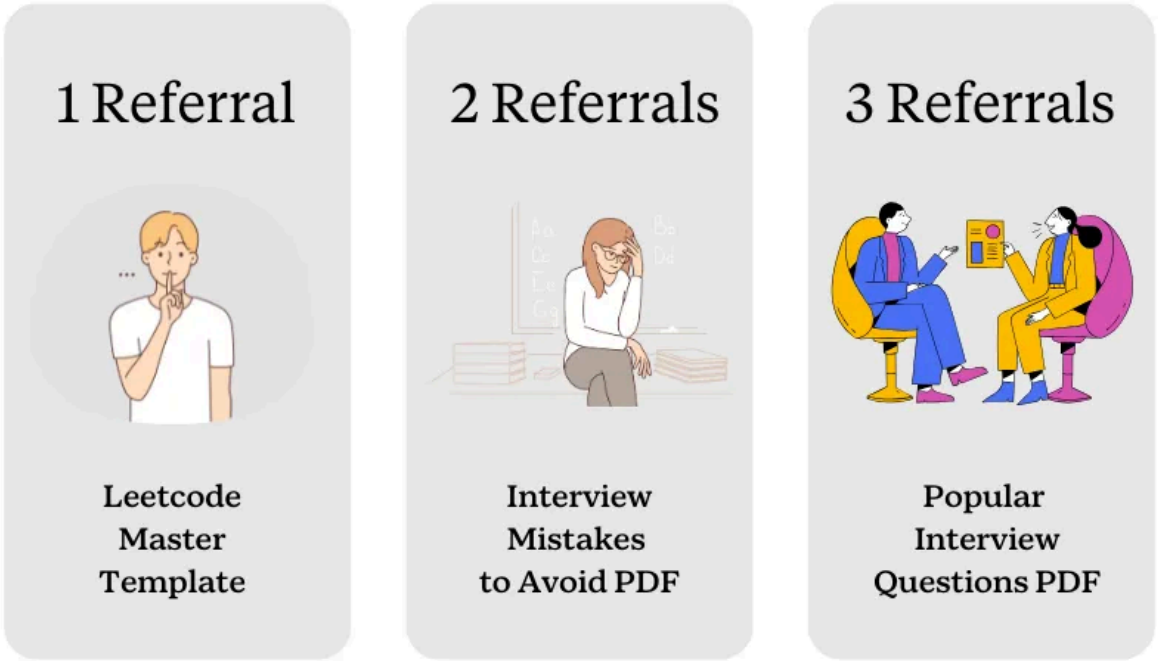


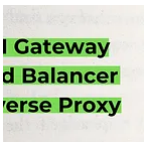
👋 **Find me on LinkedIn | Twitter | Threads | Instagram**

Want to advertise in this newsletter? 📰

If your company wants to reach a 170K+ tech audience, advertise with me.

Thank you for supporting this newsletter. Consider sharing this post with your fri and get rewards. Y'all are the best.

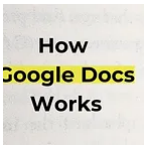




API Gateway vs Load Balancer vs Reverse Proxy 🌟

NEO KIM • 29 AUG

Read full story →



How Does Google Docs Work 🔥

NEO KIM • 22 AUG

Read full story →

References

- How to Design a Good API and Why it Matters
- REST API Design Best Practices Handbook
- Atlassian REST API design guidelines
- API versioning
- Rate Limiting: A Useful Tool with Distributed Systems
- Page-based pagination replaced by cursor-based pagination
- Best practices for RESTful web API design
- Block diagrams created with Eraser



128 Likes · 15 Restacks

← Previous Ne

Discussion about this post

Comments Restacks



Write a comment...



Aman 9 Sept

Liked by Neo Kim

Excellent content 👏

LIKE (1) REPLY

1 reply by Neo Kim



Adam 24 Sept

I do not get the preface. What does that have to do with the rest of the article? You can build monolith that uses REST APIs internally or externally. That doesn't automatically solve scaling modularity issues. The technology (REST) is orthogonal to the architecture (monolith vs micro You can build scalable microservices using non-REST protocols (e.g. message queues, GRPC, e buses). So REST is not a requirement for scalability.

LIKE (1) REPLY

4 more comments...

© 2025 Neo Kim · Publisher Privacy

Substack · Privacy · Terms · Collection notice
Substack is the home for great culture